Pocket article: How to implement and use `.noinit` RAM

23 Nov 2021 by **Noah Pendleton**

Imagine there's an embedded system that needs to persist some state when the processor restarts (either intentionally or due to a catastrophic error).

This could be some external hardware information (what's the position of a motor or actuator?) or a method to communicate the reset to the user (display some information on the device's display).

A simple way to store information through a reboot is to use what's called "non-initialized" memory.

> *Note: this article applies to systems with Static Random-Access Memory (SRAM); systems with DRAM will likely need alternate strategies!*

This pocket article will describe how a non-initialized region of memory works, how to implement it, and how it can be used in a typical embedded system.

Table of Contents

Typical memory regions in an embedded program

For more background, see these references on linker scripts:

- From Zero to main(): Demystifying Firmware Linker Scripts
- Stargirl Flower's *outstanding* "The most thoroughly commented linker script (probably)" post
- Elecia White's (of embedded.fm !) excellent Memory Map talk

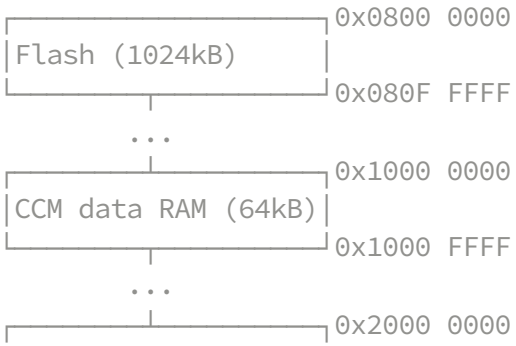
A typical small embedded device (running bare-metal or an RTOS) will usually have two types of memory available:

- read-only memory (usually flash)
- read-write memory (usually SRAM)

For example, the STM32F407VG chip has:

- 1 megabyte of flash
- 192 kilobytes of SRAM

On that chip, flash and SRAM are mapped to the following addresses:



```
|SRAM (128kB)       |
└-------------------┘0x2001 FFFF
```

Memory sections in an embedded program

A simple embedded application targeting the STM32F407VG would usually have these output sections in its memory map:

```
❯ arm-none-eabi-size -Ax build/main.elf
build/main.elf  :
section               size         addr
.text               0x1314    0x8000000
.data                 0x78   0x20000000
.bss                  0xdc   0x20000078
```

Where:

- `.text` contains **read-only** data, such as executable code or `const` data
- `.data` contains *statically initialized* **read-write** data (variables that have a non-zero initialization value)
- `.bss` contains *zero-initialized* **read-write** data

When our program starts, the `.data` section will be loaded from the *load address* (LMA), and the `.bss` section will be set to all zeros, as part of the program's startup code.

Implementing a `.noinit` section

If we want a section of RAM that is *not* initialized on startup, we can specify a dedicated region and output section for it. The following example is for GNU `ld`-compatible linker scripts (applies to GNU `ld` and LLVM `lld`, and toolchains based on those).

```
MEMORY
{
  FLASH  (rx)  : ORIGIN = 0x08000000, LENGTH = 1M
  RAM    (rwx) : ORIGIN = 0x20000000, LENGTH = 128K - 0x100
  /* Put a noinit region at the top 256 bytes of RAM  */
  NOINIT (rwx) : ORIGIN = 0x20000000 + 128K - 0x100, LENGTH = 0x100
}

SECTIONS
{
  ...
  .noinit (NOLOAD):
  {
```

```
    /* place all symbols in input sections that start with .noinit */
    KEEP(*(*.noinit*))
  } > NOINIT
  ...
}
```

Note that we're using the arithmetic support in ld to compute the origin of the `NOINIT` region; purely optional, the start positions could easily be hard-coded too (and might be preferable, since it would be more explicit! see https://www.python.org/dev/peps/pep-0020/)

Now any symbol that's placed into a `.noinit*` section will be located in the specified region, and *will not* be initialized by our program's startup code!

To place a symbol into a `.noinit` region, see the following C code examples:

```
// For GCC or Clang or derived toolchains, use the "section" __attribute__ .
__attribute__((section(".noinit"))) volatile int my_non_initialized_integer;

// for IAR EWARM, it varies, but typically:
__no_init volatile int my_non_initialized_integer @ ".noinit";
```

Note that it may be necessary to mark the variables as "volatile" (as in the example above) to ensure the compiler doesn't optimize away the "store" instructions! when in doubt, check the generated assembly where the variable is accessed (for example, by running gdb with the .elf, and using the `disassemble /s <function>` command) If your system does require volatile, it might be worth adding a comment explaining why, since it's usually quite rare that volatile is needed.

We can verify that our symbol ended up in the correct location by looking at the `.map` file (add `-Wl,-Map=app.map` to the linker flags):

```
.noinit         0x000000002001ff00       0x04
 *(*.noinit*)
 .noinit        0x000000002001ff00       0x04 build/src/main.o
                0x000000002001ff00                my_non_initialized_integer
```

We can also look at our binary with the `size` binutil and see the new section:

```
❯ arm-none-eabi-size -Ax build/main.elf
build/bootloader.elf  :
section                size         addr
.text                0x1314    0x8000000
.data                  0x78   0x20000000
.bss                   0xdc   0x20000078
.noinit                0x10   0x2001ff00
```

What about bootloaders

See some background information on bootloader operation here:

- How to write a bootloader from scratch (The Interrupt)

Since a bootloader usually will use the same RAM regions as the application, we need to make sure that the `NOINIT` region in the bootloader is similarly reserved from its normal `RAM` region. This can be done by simply matching the same `REGIONS` as in the application, ensuring nothing is placed into the `NOINIT` regions:

```
MEMORY
{
  FLASH  (rx)  : ORIGIN = 0x08000000, LENGTH = 1M
  RAM    (rwx) : ORIGIN = 0x20000000, LENGTH = 128K - 0x100
  /* the application relies on this noinit region being left alone! */
  NOINIT (rwx) : ORIGIN = 0x20000000 + 128K - 0x100, LENGTH = 0x100
}
```

Some practical examples

Let's take a look at some simple examples showcasing how a `.noinit` section can be implemented and used

No bootloader, just application

(See How to write a bootloader from scratch (The Interrupt): Message passing to catch reboot loops for another example of this!)

This system just has a single application that immediately starts when the chip is powered up. There's two variables located in the `.noinit` section:

```
// Two non-initialized variables, used to demonstrate keeping information
// through chip reset. Marked 'volatile' to ensure the compiler doesn't optim
// away the STR's to these addresses
#define RESET_MAGIC 0xDEADBEEF
// magic value used to check if the variables are initialized
__attribute__((section(".noinit"))) volatile uint32_t reset_count_magic;
// reset counter, incremented on every warm reset
__attribute__((section(".noinit"))) volatile uint32_t reset_count;
```

When the chip is initially powered on, the contents of SRAM is unknown. To handle this, the `reset_count_magic` variable contains a special value when initialized. To use it, we might do something like this:

```
if (reset_count_magic != RESET_MAGIC) {
  reset_count_magic = RESET_MAGIC;
  reset_count = 0;

  printf("First reset!\n");
}

printf("Reset count: %lu\n", ++reset_count);
```

After a cold power on, the `reset_count_magic` should persist through warm resets (eg if a Hard Fault happens, or the system intentionally reboots), and the `reset_count` should increment.

You can see more details in the implementation here:

https://github.com/noahp/cortex-m-bootloader-sample/tree/app-only

Bootloader and application, separate linker scripts

This system has a bootloader and application, placed into separate pages of flash memory:

```
MEMORY
{
  BOOTLOADER_FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 16K
  APP_FLASH (rx) : ORIGIN = 0x08004000, LENGTH = 1M - 16K
  RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K - 0x100
  NOINIT (rwx) : ORIGIN = 0x20000000 + 128K - 0x100, LENGTH = 0x100
}
```

Each linker script specifies the correct region to place read-only code and data, for example, for the bootloader:

```
SECTIONS
{
    .text :
    {
        KEEP(*(.isr_vector))
        *(.text*)
        KEEP(*(.init))
        KEEP(*(.fini))
        *(.rodata*)
    } > BOOTLOADER_FLASH
}
```

On chip power up, the bootloader runs first (since it's placed at the lowest flash address). In this example, the bootloader and app share a source file containing the non-initialized variable:

```
// noinit.c

#include <stdint.h>

// marked 'volatile' to ensure the compiler doesn't optimize away the STR's to
// these addresses
__attribute__((section(".noinit"))) volatile uint32_t mailbox[4];
```

The bootloader can set values into that variable, and the application can read them:

```
// bootloader main.c:
  extern volatile uint32_t mailbox[4];
  mailbox[0] = get_random_number();
  printf("Set random value to mailbox: 0x%08" PRIx32 "\n", mailbox[0]);

// app main.c:
  extern volatile uint32_t mailbox[4];
  printf("mailbox was: 0x%08" PRIx32 "\n", mailbox[0]);
```

The application could also set values into the mailbox, then jump to the bootloader (eg via reset). This might be used to command the bootloader to reflash the application, for example in a "dual-bank" (aka A/B) partition scheme.

You can see more details in the implementation here:

https://github.com/noahp/cortex-m-bootloader-sample/tree/two-linker-scripts

Bootloader and application, shared linker script

This system is very similar to the one above, except the bootloader and application use the same linker script:

```
MEMORY
{
  BOOTLOADER_FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 16K
  APP_FLASH (rx) : ORIGIN = 0x08004000, LENGTH = 1M - 16K
  RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K - 0x100
  NOINIT (rwx) : ORIGIN = 0x20000000 + 128K - 0x100, LENGTH = 0x100
}

SECTIONS
{
  .text :
  {
        KEEP(*(.isr_vector))
        *(.text*)
        KEEP(*(.init))
        KEEP(*(.fini))
        *(.rodata*)
  } > FLASH__  /* placeholder region identifier */
}
```

To link the bootloader or application, the shared linker script is run through the C preprocessor to generate the linker script used for linking. The `FLASH__` placeholder is replaced with the appropriate `BOOTLOADER_FLASH`/`APP_FLASH` region for the application being linked:

```
❯ gcc -DFLASH__=APP_FLASH -E -P -C -x c-header src/common/stm32f407.ld > build
```

(A similar technique is used by the Zephyr RTOS build system to derive the correct linker script for different targets/memory configurations).

You can see more details in the implementation here:

https://github.com/noahp/cortex-m-bootloader-sample/tree/shared-linker-script

Other considerations

Backup RAM

Some chips will have a “backup RAM” or dedicated USB or Ethernet memory banks. These memories can be a convenient spot to place noinit data (if the memory is not being used for another purpose).

For example, on the STM32F407VG, there's a 4kB "Backup RAM" intended for very low power sleep data retention; however, the memory can be used for any purpose.

```
MEMORY
{
  /* on this chip, there's 4kB of "Backup RAM" mapped to this address */
  NOINIT (rwx) : ORIGIN = 0x40024000, LENGTH = 4k
}
```

On this chip, the backup RAM does need to be powered up to be used:

```
//! Using the stm32f407xx.h CMSIS register structures to enable the backup SRA
static void enable_backup_sram(void) {
  // enable power interface clock
  RCC->APB1ENR |= RCC_APB1ENR_PWREN;

  // enable backup SRAM clock
  RCC->AHB1ENR |= RCC_AHB1ENR_BKPSRAMEN;

  // enable backup SRAM
  PWR->CR |= PWR_CR_DBP;
}
```

Then we can read/write variables that are placed in that region! Since this RAM may not be typically used for the main application, this could be considered "free" memory for various purposes (such as non-initialized data!)!

Retrofitting a noinit region when you can't update the bootloader

This scenario could happen if the application needs a `.noinit` region, but the bootloader is not updateable (either due to external requirements, permanently enabling write protect on the bootloader flash pages, or avoiding the risk of potentially bricking a device if the bootloader update fails).

If the bootloader is set up to use all of RAM, you'll need to examine which locations are actually used. For example, stack may start at the top of RAM and grow down; if there's a known limit to the stack, a `.noinit` region could be added below the stack region, and the bootloader won't overrwrite it.

To illustrate this, here's a typical RAM layout for an embedded system, including stack and heap, from lowest to highest memory address:

```
.data
.bss
.heap (end of .bss, grows up)
<possible spot for retrofitted .noinit>
.stack (end of RAM, grows down)
```

Of course, if you're lucky, there may be a small reserve of RAM (backup RAM) that can be used instead!

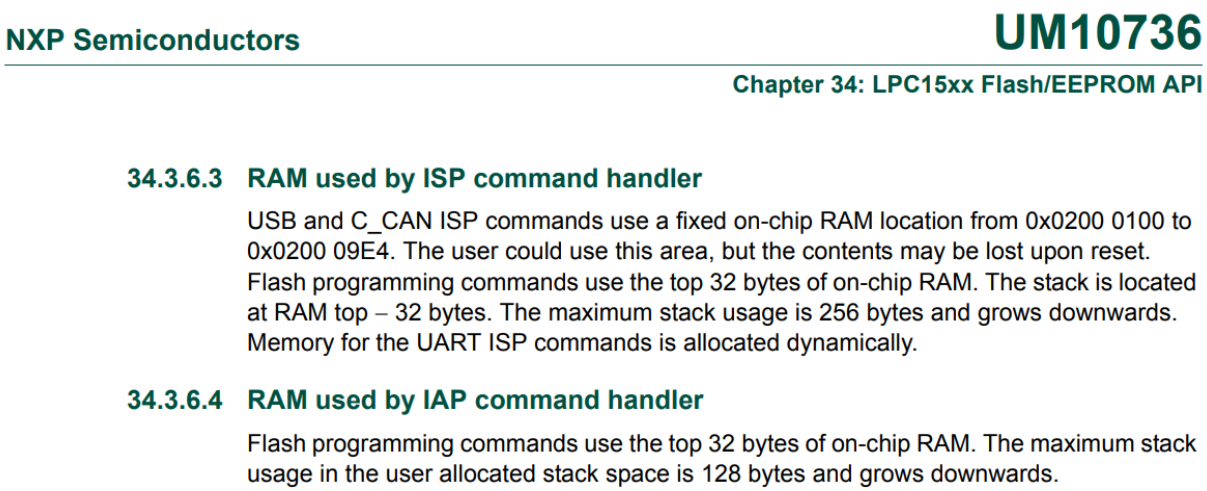
Gotchas: Watch out for ROM bootloaders (they use RAM too)

Some chips have ROM bootloaders (also referred to "ISP", In-System Programming), that can:

- run before the user application is started
- run on-demand, depending on boot pins, or programmatically launched from software

These ROM programs will often use RAM for their own purposes! which can impact `.noinit` regions. Consult the chip documentation, but be wary. Some chips don't have good documentation on the ROM bootloader.

Some chips use routines stored in ROM for other purposes (flash programming, built-in USB operations, etc); for example, the LPC15xx series documents this:



NXP Semiconductors

UM10736

Chapter 34: LPC15xx Flash/EEPROM API

34.3.6.3 RAM used by ISP command handler

USB and C_CAN ISP commands use a fixed on-chip RAM location from 0x0200 0100 to 0x0200 09E4. The user could use this area, but the contents may be lost upon reset. Flash programming commands use the top 32 bytes of on-chip RAM. The stack is located at RAM top – 32 bytes. The maximum stack usage is 256 bytes and grows downwards. Memory for the UART ISP commands is allocated dynamically.

34.3.6.4 RAM used by IAP command handler

Flash programming commands use the top 32 bytes of on-chip RAM. The maximum stack usage in the user allocated stack space is 128 bytes and grows downwards.

Test implementation

Some chips can lose SRAM contents when resetting (some chips wire up `NVIC` reset to pulse the physical reset line!), which can make these schemes unstable and a pain to debug.

When bringing up a new chip, it's worth consulting the docs on this topic, as well as doing a quick check that SRAM contents persist through chip reset. This can be done by:

1. writing a pattern to entire reserved area (eg `0x01234567`)
2. reset the chip and confirm the pattern is still valid across the region

Further Reading

- https://mcuoneclipse.com/2014/04/19/gnu-linker-can-you-not-initialize-my-variable/
- https://mcuoneclipse.com/2013/04/14/text-data-and-bss-code-and-data-size-explained/
- https://atadiat.com/en/e-how-to-preserve-a-variable-in-ram-between-software-resets/

***Noah Pendleton** is an embedded software engineer at Memfault. Noah previously worked on embedded software teams at Fitbit and Markforged*